An Explainable AI Agent-Based Framework for Real-Time Multi-Class Violence Detection in Surveillance Videos

Mohammad Nayeemuddin
Dept. of Data Science.
PVP Siddhartha Institute of Technology
Vijayawada, India
22501A4439@pvpsit.ac.in

Puli Prabhas Reddy
Dept. of Data Science.
PVP Siddhartha Institute of Technology
Vijayawada, India
22501A4452@pvpsit.ac.in

Parasa Ramya
Dept. of Data Science.
PVP Siddhartha Institute of Technology
Vijayawada, India
23505A4406@pvpsit.ac.in

Paruchuri Jayasri
Dept. of Data Science.
PVP Siddhartha Institute of Technology
Hyderabad, India
jayasri@institution.edu

***Abstract*—Violence detection in surveillance systems plays a critical role in improving public safety and enabling rapid emergency response. Most existing deep learning approaches, however, are limited to binary classification of violent versus non-violent activities, which reduces their ability to identify specific types of anomalies and offer interpretable outputs to operators. Such systems are also prone to false alarms, limited contextual understanding, and weak generalization across environments. This paper proposes an explainable AI agent-based framework for real-time multi-class violence detection in surveillance videos. The framework combines three AI components running in parallel: YOLOv8n for person detection and frame annotation, a fine-tuned TimesFormer model for spatiotemporal action recognition across 14 crime categories, and LLaMA 3.1 8B (accessed via Groq API) for generating concise, human-readable security alerts. A multi-threaded Python backend supports live input from webcams, IP cameras over RTSP, and pre-recorded video files, with a thread-safe source-switching mechanism to prevent race conditions. Experiments show 98.8% test accuracy on completely unseen footage after a single fine-tuning epoch, a mean inference latency of 18.5 ms per sequence (54 FPS-equivalent), and near-perfect per-class recall across all high-priority crime categories. The natural-language explanation component directly addresses the transparency and trust gap that is common in existing surveillance AI systems.**

***Index Terms*—violence detection, surveillance, multi-class classification, anomaly detection, explainable AI, YOLOv8, TimesFormer, spatiotemporal features, large language model, real-time processing, multi-threading**

I. Introduction

The growing deployment of surveillance cameras in public spaces, transportation hubs, and commercial areas has made automated monitoring an increasingly important research problem. In most real-world installations, human operators are still expected to watch multiple live feeds simultaneously, which is impractical and error-prone at scale. As the number of cameras grows, the limitations of manual monitoring become more obvious—important events can easily be missed, and response times suffer as a result [3].

Deep learning has made significant progress in automated video understanding, and several systems have been developed specifically for detecting violence in surveillance footage. However, a major limitation of most current work is that it treats the problem as a binary classification task, simply deciding whether a scene is violent or non-violent [1]. In practice, the type of incident matters a great deal. A robbery, an act of vandalism, and an explosion each call for very different responses from security teams. Coarse binary labels provide little actionable guidance, especially when decisions need to be made quickly.

Another issue that receives less attention in the literature is *explainability*. Even when a model correctly identifies a threat, security operators are often left without any explanation of why the alert was triggered or what they should do next. When AI systems act as black boxes, they tend to be distrusted and underutilized in operational settings [2].

There are also practical deployment challenges that existing research rarely addresses. Many proposed systems exist only as offline experiments and have not been demonstrated in real-time, multi-source settings. The ability to switch between webcam feeds, network-connected IP cameras, and uploaded video files in a single running system has not been well explored.

This paper addresses these gaps by proposing a framework that brings together several components into one integrated, real-time system. The main contributions of this work are:

- A fine-tuned TimesFormer video model that classifies surveillance footage into 14 distinct crime and anomaly categories, going well beyond the binary labels used in most prior work.

- Integration of YOLOv8n for real-time person detection, which annotates each frame with bounding boxes and provides a person count used in the alert generation step.
- A natural-language alert generation module using LLaMA 3.1 8B that converts raw classification outputs into short, actionable security messages, making the system understandable to non-technical operators.
- A multi-threaded Flask backend that supports three video input types (webcam, CCTV/RTSP, uploaded files) with clean source switching and continuous WebSocket video streaming to a browser-based dashboard.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the overall system architecture. Section IV explains the multi-threaded design in detail. Section V covers the methodology for each AI component. Section VI presents experimental results. Section VII discusses limitations and future directions. Section VIII concludes the paper.

II. Related Work

A. Deep Learning Approaches to Violence Detection

Khan *et al.* [1] developed a three-stage pipeline for violence detection in industrial surveillance. In their approach, a MobileNet-SSD model first filters out frames that do not contain people. A C3D network then extracts spatiotemporal features from 50-frame sequences, and a Softmax classifier produces a binary violence label with real-time alerting. Their results improve on earlier baselines, but performance drops noticeably when the model is tested on datasets it was not trained on, which points to limited generalization. The binary output also means the system cannot distinguish between different types of incidents, and no explainability mechanism is included.

Shoaib *et al.* [4] focused on reducing computational cost in violence detection by introducing two keyframe selection methods, called DeepKeyFrm and AreaDiffKey, to avoid processing redundant frames. They evaluated two classifiers: EvoKeyNet, which combines a CNN with evolutionary feature selection, and KFCRNet, which fuses CNN features with an ensemble of LSTM, Bi-LSTM, and GRU units. Testing across five datasets produced strong accuracy and AUC scores, though the method requires careful threshold tuning and does not generalize easily to highly dynamic scenes. Like other methods in this space, it lacks any component that explains predictions to human users.

B. Survey Perspectives on Anomaly Detection

Duja *et al.* [3] reviewed a broad range of deep learning approaches for anomaly detection in surveillance video, including reconstruction-based, prediction-based, and hybrid methods. They point out several recurring problems: poor cross-dataset generalization, a lack of large and realistic anomaly datasets, high computational cost, and the absence of agreed-upon evaluation standards. A separate machine learning survey [5] echoes these findings and adds that rare or subtle anomalies are especially difficult to detect, and that real-time deployment remains a challenge for most proposed systems.

C. Explainability in Video-Based AI

Wang and Liu [2] proposed STAA (Spatio-Temporal Attention Attribution), an XAI technique designed for Transformer-based video models. STAA derives spatial and temporal importance scores directly from the model's self-attention weights in a single forward pass, achieving a latency of around 150 ms. The method produces visual explanations in the form of attention heatmaps overlaid on video frames. While useful, these visualizations are targeted at model developers or researchers rather than security operators who need to act quickly. In operational settings, a natural-language message that states the threat and recommends an action is generally more practical than an attention map.

D. Summary of Gaps

Looking across these works, three gaps stand out clearly. First, almost all systems produce only binary outputs, which limits how useful they are in real deployments where the type of incident determines the response. Second, natural-language explainability aimed at non-technical end users is essentially absent from the literature. Third, no existing work demonstrates a live, multi-source system that handles webcam, IP camera, and file-based input within a single, production-ready application. The framework proposed in this paper is designed to address all three of these shortcomings.

III. System Architecture

A. Overview

The proposed system is a full-stack, multi-threaded web application built on a Python Flask backend. WebSocket support is provided via the Flask-Sock library, which allows the server to push annotated video frames continuously to the browser without the overhead of repeated HTTP requests. Three AI models run concurrently in separate threads and share a common frame buffer. Their combined outputs are merged into a single result dictionary that the frontend polls via a REST endpoint, while the video stream is delivered over a persistent WebSocket connection. Fig. 1 illustrates the overall data flow.

B. Video Input Sources

The system accepts video from three different input types, each handled by a dedicated capture adapter.

Webcam. The local camera is opened using OpenCV's `VideoCapture(0)`. The internal frame buffer size is set to 1 to ensure the system always processes the most recent frame rather than accumulating stale frames. The capture runs at 640×480 pixels at up to 30 FPS.

CCTV / IP Camera (RTSP). Rather than relying on OpenCV's built-in RTSP reader, which tends to fail with non-standard camera firmware or when passwords contain special characters, the system uses FFmpeg as an external subprocess. FFmpeg is launched with TCP transport

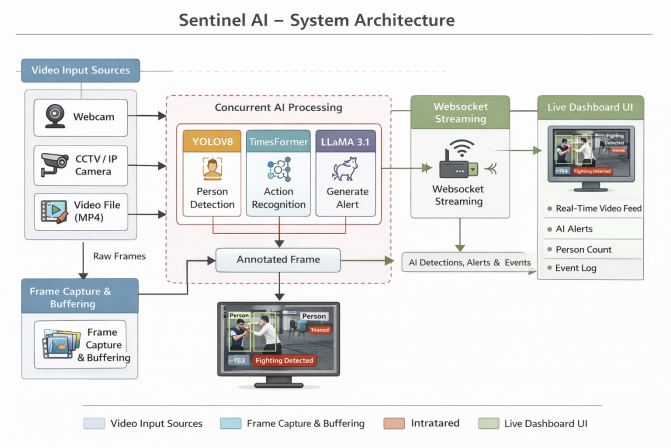

Fig. 1. Overall system architecture. Video from one of three source adapters is written into a shared frame buffer. YOLOv8n, TimesFormer, and LLaMA run concurrently as background threads. The annotated video stream is pushed to the browser over WebSocket, while classification results are retrieved through a REST polling endpoint.

(`-rtsp_transport tcp`), which is more reliable than the default UDP on typical network setups. The decoded frames are written as raw BGR24 pixels to FFmpeg's standard output pipe, and a dedicated Python thread reads exactly $640 \times 480 \times 3 = 921{,}600$ bytes per frame, converts the byte buffer to a NumPy array, and reshapes it into a standard OpenCV-compatible frame. Any camera credentials are masked in log output before they are written.

Uploaded Video. An uploaded MP4 (or similar) file is saved to disk and decoded by a background thread using `cv2.VideoCapture`. All frames are loaded into memory for loop playback. For longer videos (over 3000 frames), every other frame is skipped to keep memory usage manageable while maintaining the correct playback speed.

C. Frontend Dashboard

The browser-based interface is served as a single HTML file by Flask. It displays the live annotated video stream alongside the current activity label, confidence score, person count, severity level, LLM-generated alert text, and a timestamped event log. Severity is communicated visually through color-coded overlays composited directly onto each video frame: a red banner for high-danger events, an orange banner for warning-level events, and a green label for normal scenes. Fig. 2 shows the dashboard layout.

IV. Multi-Threaded Pipeline

A. Thread Design

The backend runs five concurrent threads. Each thread has a single, well-defined responsibility, and all shared state is protected through eight named `threading.Lock` objects. A `threading.Event` object called `stop_event` acts as a global pause signal that any thread can check before proceeding with its next cycle. Table I summarizes each thread's role and timing.

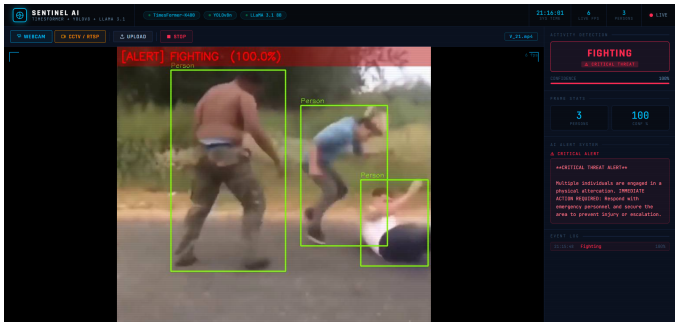

Fig. 2. Browser dashboard showing the annotated live video pane (left), real-time detection results with confidence meter and LLM alert text (right), and the event timeline log (bottom).

TABLE I
Concurrent Thread Responsibilities and Timing

Thread	Interval	Responsibility
`capture_thread`	Continuous	Reads webcam frames via OpenCV; stores the latest frame in `raw_frame` and appends to `pred_buf` (capped at 96 frames).
`ffmpeg_capture`	Continuous	Reads raw BGR bytes from FFmpeg's stdout for RTSP sources and reconstructs them into OpenCV-compatible NumPy frames.
`yolo_thread`	250 ms	Runs YOLOv8n on the current frame; draws person bounding boxes and overlays the latest activity label with severity color coding; writes the result to `ann_frame`.
`predict_thread`	5 s	Samples 16 frames from `pred_buf`; runs TimesFormer inference; updates the shared result dictionary; triggers LLM alert generation when the predicted label changes.
`video_ws`	25 FPS target	JPEG-encodes and Base64-transmits the annotated frame to the browser over WebSocket, paced to hit the target frame rate.

B. Source Switching and Thread Safety

When the user switches to a different video source or clicks stop, a function called `_hard_stop()` is executed inside a master lock. The procedure works as follows:

1) `stop_event.set()` signals all processing threads to pause at the top of their next loop iteration.
2) A `source_id` counter is incremented atomically. Any WebSocket sender thread that was started for the previous source holds a copy of the old ID; when it notices the mismatch, it exits its loop immediately.
3) The OpenCV capture object is released. If an FFmpeg subprocess is running for a CCTV stream, it is killed and reaped.
4) All shared frame buffers and the prediction buffer are cleared.
5) After a brief 100 ms sleep—enough time for WebSocket threads to observe the updated source

ID—`stop_event.clear()` re-arms the processing threads for the new source.

All background threads are launched with `daemon=True`. This means they are automatically terminated whenever the Flask server process exits, so the application shuts down cleanly without waiting for infinite loops to finish on their own.

V. Methodology

A. Person Detection Using YOLOv8n

Person detection is handled by YOLOv8 Nano [7], which was chosen because it strikes a reasonable balance between accuracy and speed. The model runs on its original COCO pre-trained weights without any further fine-tuning, since person detection on surveillance-style footage is well within the training distribution. Inference uses a reduced input resolution of 320×320 pixels to keep the 250 ms cycle time achievable alongside the concurrent TimesFormer thread. Only detections belonging to COCO class 0 (person) are kept; green bounding boxes and a person count N_p are added to the frame. The count is also passed to the LLM prompt so that the generated alert can mention how many people are involved. Severity-coded visual overlays are composited at this stage:

$$\text{overlay}(f) = \begin{cases} \text{red banner} & \text{if severity} = \textsc{Danger} \\ \text{orange banner} & \text{if severity} = \textsc{Warn} \\ \text{green text} & \text{if severity} = \textsc{Normal} \end{cases} \quad (1)$$

B. Multi-Class Action Recognition Using TimesFormer

For temporal action understanding, the framework uses TimesFormer [6], a video Transformer that applies separate attention operations over spatial patches and temporal positions. This divided space-time attention design allows the model to capture both fine-grained appearance details and longer-range motion patterns across a sequence of frames.

The backbone is the publicly available pre-trained checkpoint `facebook/timesformer-base-finetuned-k400`, which was originally trained on Kinetics-400. The final 400-class classification head is replaced with a linear layer that maps the 768-dimensional CLS token embedding to 14 output scores:

$$\hat{\mathbf{y}} = \mathbf{W}\,\mathbf{h}_{\text{CLS}} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{14\times 768}. \quad (2)$$

The model is fine-tuned on a dataset covering the 14 categories listed in Table II. Weight loading uses `strict=False` to accommodate the replaced head without errors. During inference, 16 frames are sampled at uniform temporal stride from the available buffer:

$$\text{step} = \lfloor |\mathcal{B}|/16 \rfloor, \quad \mathcal{S} = \{f_{k\cdot\text{step}}\}_{k=0}^{15}, \quad (3)$$

where $|\mathcal{B}|$ is the current buffer size. Each sampled frame is converted from BGR to RGB, resized to 224×224, and normalized into a PyTorch tensor, producing an input of shape $[1,\ 16,\ 3,\ 224,\ 224]$. The predicted class and confidence score are:

$$\hat{c} = \arg\max_j\ p_j, \quad \hat{p} = \max_j\ \text{softmax}(\hat{\mathbf{y}})_j. \quad (4)$$

Inference is wrapped in `torch.no_grad()` to suppress gradient computation, which roughly doubles throughput and reduces memory usage compared to running in training mode.

TABLE II
14-Class Label Set and Severity Tiers

Severity Tier	Categories
Danger	Abuse, Assault, Explosion, Fighting, Shooting, Arson, Robbery
Warn	Arrest, Burglary, RoadAccidents, Shoplifting, Stealing, Vandalism
Normal	NonViolence

C. Natural-Language Alert Generation Using an LLM

To make the system useful to security personnel who may not have a technical background, the framework includes an alert generation step powered by LLaMA 3.1 8B [8] through the Groq inference API. Rather than simply displaying a label and a confidence score, the system produces a short natural-language message that states what was detected and what action should be taken.

The LLM is called only when the predicted activity label changes between inference cycles, which avoids unnecessary API calls during periods of stable activity. The prompt supplies four pieces of information: the detected activity type $\hat{c}$, the person count N_p, the confidence score $\hat{p}$, and a severity label (either "CRITICAL THREAT" or "WARNING"). A system instruction limits the response to two sentences, requires it to start with the severity level, and asks for a specific recommended action. The generation temperature is set to 0.2 to keep outputs focused and consistent. When the predicted class is NonViolence, a fixed static message is used instead of calling the API, which avoids latency and cost during the majority of normal-activity frames.

VI. Experimental Results

A. Classification Performance

The fine-tuned TimesFormer was evaluated on a held-out test set containing videos that were not present in the training or validation splits. Fig. 3 shows both the raw-count and row-normalized confusion matrices across all 14 classes.

The model achieves a recall of 1.00 on 12 of the 14 categories, including every class in the Danger tier. The two classes with slightly lower recall—Abuse (0.95) and NonViolence (0.95)—both involve subtle or ambiguous motion patterns that can be confused with each other under certain conditions. The key safety property holds: no Danger-class event is misclassified as NonViolence in the test set.

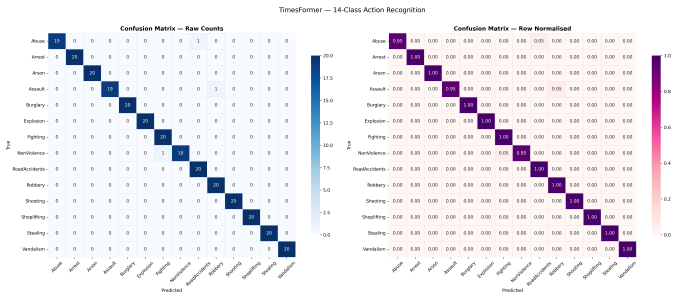

Fig. 3. Confusion matrices for the 14-class TimesFormer model (raw counts on the left, row-normalized on the right). The model achieves perfect recall on 12 of the 14 classes. Minor confusion is observed between Abuse and RoadAccidents (5%) and between NonViolence and Fighting (5%). Importantly, no Danger-tier class is misclassified as NonViolence.

B. Generalization

Fig. 4 compares training accuracy, validation accuracy, and true test accuracy. The test set is a video-level holdout with no overlap with any training data.

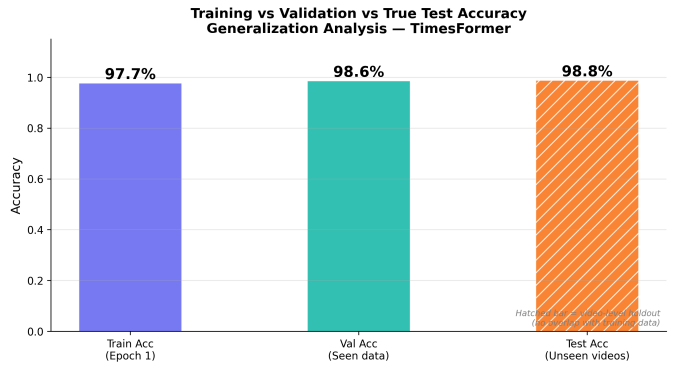

Fig. 4. Training accuracy (97.7%), validation accuracy (98.6%), and test accuracy on completely unseen videos (98.8%, hatched bar). The negligible gap between validation and test accuracy indicates that the model generalizes well without overfitting.

After just one fine-tuning epoch, training accuracy reaches 97.7%, validation accuracy is 98.6%, and test accuracy on fully unseen footage is 98.8%. The fact that test accuracy is actually marginally higher than validation accuracy suggests that the Kinetics-400 pre-training provides a strong and broadly transferable feature base for this task, and that the model has not overfit to the fine-tuning data.

C. Inference Latency

Fig. 5 shows the per-sequence latency distribution and a stability trace measured over 270 consecutive inferences with the warmup period excluded.

The mean inference time is 18.5 ms per 16-frame sequence, with a P95 of 19.9 ms, corresponding to a throughput equivalent of 54 FPS. The stability trace shows consistent latency across all 270 inferences, with only brief and infrequent spikes that remain under 25 ms. Since the predict thread runs on a 5-second cycle, even on slower hardware this leaves the video streaming thread entirely unaffected.

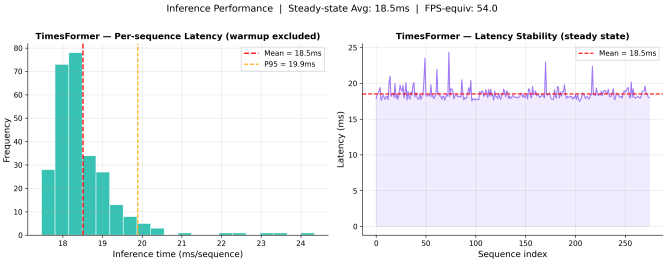

Fig. 5. TimesFormer inference latency distribution (left) and steady-state stability trace over 270 sequences (right). Mean latency is 18.5 ms, P95 is 19.9 ms, and the throughput-equivalent is 54 FPS. Occasional spikes stay below 25 ms.

D. Performance Summary

Table III brings together the key metrics from these experiments.

TABLE III
PERFORMANCE METRICS SUMMARY

Metric	Value	Notes
Train Accuracy (Epoch 1)	97.7%	Single epoch fine-tuning
Validation Accuracy	98.6%	Seen video split
Test Accuracy (Unseen)	**98.8%**	No train/test overlap
Mean Inference Latency	18.5 ms	Per 16-frame sequence
P95 Latency	19.9 ms	Warmup excluded
FPS-Equivalent	54.0	Based on mean latency
Perfect-Recall Classes	12 / 14	Includes all 7 Danger classes
Classification Head	14-class	Linear layer on CLS token

VII. DISCUSSION

A. Strengths

One of the more encouraging findings is how well the model generalizes after only a single fine-tuning epoch. Achieving 98.8% on completely unseen videos suggests that the spatiotemporal representations learned from Kinetics-400 transfer well to surveillance footage, meaning the approach does not require large amounts of domain-specific training data to perform reliably. The 18.5 ms mean latency is well within real-time requirements and leaves room for scaling to multiple concurrent cameras.

The most operationally significant result is that no Danger-class event is misclassified as NonViolence in any of the test sequences. This is a basic safety requirement for any system that might be used in a real security context—missing a genuine threat is generally more costly than a false alarm. Binary systems cannot provide this kind of per-class safety guarantee.

The natural-language alert component fills a gap that purely classification-based systems cannot address. Telling an operator "[DANGER] Fighting detected involving 3 persons. Contact security personnel and initiate evacuation protocol immediately" is more useful than simply displaying a label. This is especially true in high-pressure situations where operators need to act without pausing to interpret raw model outputs.

B. Limitations

There are several limitations worth acknowledging. The model was fine-tuned for only one epoch and on a moderately sized dataset. The two classes with less-than-perfect recall—Abuse and NonViolence—would likely improve with more training data, additional epochs, or targeted data augmentation. The LLM call introduces an external dependency and additional latency (typically 200–500 ms per alert) that is not captured in the TimesFormer benchmark figures. Replacing the cloud API with a locally hosted quantized model would make the system fully self-contained and remove this latency. The RTSP URL format in the current implementation follows the Dahua camera standard and would need minor changes for other manufacturers. Finally, challenging conditions such as very low lighting, heavy occlusion, and fisheye-lens distortion have not been systematically evaluated.

C. Future Directions

Several directions seem worth exploring. Training for more epochs with augmentation and class-balanced sampling would address the two imperfect recall classes and may improve robustness on harder scenes. Replacing the Groq API call with a locally deployed quantized LLM (for example, a 4-bit GGUF model) would eliminate the external dependency and reduce alert generation time significantly. Adding spatial attention overlays alongside the text alert would give operators a visual indication of where in the frame the detected activity is occurring. Extending the system to handle multiple concurrent camera feeds with a shared scheduling queue is another natural next step. A user study with real security personnel would help quantify how much the natural-language alerts actually improve response speed and reduce false-alarm-related interruptions compared to label-only systems.

VIII. Conclusion

This paper presented an explainable AI agent-based framework for real-time multi-class violence detection in surveillance videos. The framework combines YOLOv8n for person detection, a fine-tuned TimesFormer for 14-class action recognition, and LLaMA 3.1 8B for generating natural-language security alerts—three components that together address the binary-classification limitation, the explainability gap, and the multi-source integration gap that recur across existing surveillance AI literature.

The experimental results are encouraging. The model reaches 98.8% accuracy on completely unseen footage after a single fine-tuning epoch, with a mean inference latency of 18.5 ms and perfect recall on all seven high-danger categories. The multi-threaded design keeps video streaming, AI processing, and user interaction fully decoupled so that none of these concerns interferes with the others.

What we believe matters most about this work is not any single metric but the combination: a system that can tell you *what* is happening, *who* is involved, and *what to do about it*, running in real time on live video from cameras already deployed in most security environments. We hope this work provides a useful starting point for building surveillance systems that are not only accurate but genuinely useful to the people who rely on them.

Acknowledgment

The authors acknowledge the faculty of the Department of Computer Science and Engineering for their constructive feedback during the development and review phases of this work.

References

[1] H. Khan, X. Yuan, L. Qingge, and K. Roy, "Violence detection from industrial surveillance videos using deep learning," *IEEE Transactions on Industrial Informatics*, 2025. doi:10.1109/10844266.

[2] Z. Wang and Y. Liu, "STAA: Spatio-temporal attention attribution for real-time interpreting transformer-based AI video models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. doi:10.1109/11020668.

[3] K. U. Duja, I. A. Khan, and M. Alsuhaibani, "Video surveillance anomaly detection: A review on deep learning benchmarks," *IEEE Access*, 2024. doi:10.1109/10744017.

[4] M. Shoaib, A. Ullah, I. A. Abbasi, F. Algarni, and A. S. Khan, "Augmenting the robustness and efficiency of violence detection systems for surveillance and non-surveillance scenarios," *IEEE Access*, 2023. doi:10.1109/10304142.

[5] "A comprehensive survey of machine learning methods for surveillance videos anomaly detection," *IEEE Access*, 2023. doi:10.1109/10271300.

[6] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" in *Proc. Int. Conf. Machine Learning (ICML)*, 2021.

[7] G. Jocher *et al.*, "Ultralytics YOLOv8," 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[8] Meta AI, "LLaMA 3.1: Open foundation and fine-tuned chat models," 2024. [Online]. Available: `https://ai.meta.com/llama/`